

P1 — AI / Agents Lead

Role: Build the core intelligence layer: Pydantic schemas, LangGraph graph, and all 5 agents.

Timeline: 3 days. Work in `backend/` only.

Files You Own

```
backend/
├─ models/schemas.py
├─ agents/graph.py
├─ agents/planner.py
├─ agents/interaction_analyzer.py
├─ agents/knowledge_retriever.py
├─ agents/risk_assessor.py
└─ agents/nba_generator.py
```

Dependencies: `langgraph`, `anthropic`, `pydantic`, `python-dotenv`, `time`, `uuid`, `typing`

Interface Contracts

You Consume (from P2)

- `backend/memory/vector_store.py` — must expose `search(query: str, collection: str, n: int = 5) -> list[dict]`
- `backend/memory/episodic.py` — must expose `get_memory_context(risk_level: str, risk_score: float) -> MemoryContext`

You Provide (to P3)

- `backend/agents/graph.py` — must expose `run(raw_input: str, account_id: str) -> RecommendationOutput`

Day 1 — Schemas + Graph + First 2 Agents

1. `backend/models/schemas.py`

Define these exact classes. Do not rename fields.

```
from pydantic import BaseModel
from typing import List, Optional, Literal
from datetime import datetime

class AgentStep(BaseModel):
    agent_name: str
    action: str
    reasoning: str
    duration_ms: int

class Evidence(BaseModel):
    id: str
    source_type: Literal["meeting_note", "playbook", "memory", "crm"]
    source_name: str
    excerpt: str
    relevance_score: float

class Action(BaseModel):
    title: str
    description: str
    confidence: float
    estimated_impact: str
    evidence_ids: List[str]
    precedent_accounts: List[str]

class MemoryContext(BaseModel):
    similar_cases_found: int
    confidence_boost: float
    base_confidence: float
    boosted_confidence: float
```

```

precedent_accounts: List[str]

class RecommendationOutput(BaseModel):
    request_id: str
    account_name: str
    csm_name: str
    renewal_date: str
    risk_score: float
    risk_level: Literal["critical", "high", "medium", "low"]
    signal_count: int
    agent_trace: List[AgentStep]
    evidence: List[Evidence]
    primary_recommendation: Action
    alternatives: List[Action]
    memory_context: Optional[MemoryContext]
    generated_at: datetime

```

2. `backend/agents/graph.py`

Set up a `LangGraph` `StateGraph` with this exact state shape:

```

state = {
    "raw_input": str,          # the meeting transcript
    "account_id": str,         # passed from FastAPI
    "signals": list[dict],     # output of interaction_analyzer
    "knowledge_chunks": list[dict], # output of knowledge_retriever
    "risk": dict,              # output of risk_assessor
    "recommendations": dict,   # output of nba_generator
    "agent_trace": list[AgentStep],
    "account_data": dict,      # customer profile loaded by graph
}

```

Nodes (in order): `planner` → `interaction_analyzer` → `knowledge_retriever` → `risk_assessor` → `nba_generator`.

Expose one function:

```

def run(raw_input: str, account_id: str) -> RecommendationOutput:
    ...

```

This function must:

- Load the customer profile from `backend/data/customer_profiles/{account_id}.json` (P2 provides these files).
- Build the initial state.
- Execute the graph.
- Assemble and return a `RecommendationOutput` object.
- Generate `request_id` with `uuid.uuid4().hex[:12]`.
- Set `generated_at = datetime.utcnow()`.

3. `backend/agents/planner.py`

```
def plan(state: dict) -> dict:  
    ...
```

- Reads `raw_input`.
- Always returns `execution_plan = ["interaction_analyzer", "knowledge_retriever", "risk_assessor", "nba_generator"]`.
- Logs one `AgentStep`: `agent_name="planner"`, `action="created_execution_plan"`, `reasoning="Standard 4-agent pipeline for customer success analysis"`, `duration_ms` measured with `time.time()`.
- Appends the step to `state["agent_trace"]` and returns updated state.

4. `backend/agents/interaction_analyzer.py`

```
def analyze(state: dict) -> dict:  
    ...
```

- Prompts Claude (Anthropic API) with the transcript.
- Returns a list of signal dicts: `{"text": str, "type": "risk|positive|neutral", "severity": "high|medium|low"}`.
- Stores them in `state["signals"]`.
- Logs an `AgentStep` with `agent_name="interaction_analyzer"`, `action="extracted_signals"`, `reasoning` summarizing what was found, and `duration_ms`.

Claude prompt template:

You are a customer success signal extractor. Read the following meeting transcript and extract signals. For each signal, return: text (exact quote or paraphrase), type (risk, positive, or neutral), and source. Return ONLY a JSON list. No markdown fences.

Transcript:
{transcript}

Day 2 — Remaining Agents + Wiring

5. backend/agents/knowledge_retriever.py

```
def retrieve(state: dict) -> dict:  
    ...
```

- Imports `backend.memory.vector_store` (P2 builds this).
- For each signal in `state["signals"]`, calls `vector_store.search(query=signal["text"], collection="knowledge_base", n=5)`.
- Deduplicate by `source_name` + `excerpt`.
- Build `Evidence` objects. Generate `id` as `ev_{index}`.
- Store in `state["knowledge_chunks"]`.
- Log `AgentStep`: `agent_name="knowledge_retriever"`, `action="retrieved_knowledge"`, `reasoning=f"Found {len(evidence)} unique chunks from knowledge base"`.

6. backend/agents/risk_assessor.py

```
def assess(state: dict) -> dict:  
    ...
```

- Prompts Claude with `signals` + `knowledge_chunks`.
- Returns structured JSON:

```
{
```

```

    "risk_score": 0.84,
    "risk_level": "critical",
    "signal_count": 4,
    "risk_factors": ["factor 1", "factor 2"]
}

```

- Maps each `risk_factor` string to the closest `Evidence.id` by semantic similarity (string overlap is fine for MVP).
- Stores result in `state["risk"]`.
- Log `AgentStep`: `agent_name="risk_assessor"`, `action="assessed_risk"`, `reasoning=f"Risk level: {level}, score: {score}"`.

Claude prompt template:

You are a customer success risk analyst. Given these signals and knowledge articles, assess customer success risk. Return ONLY JSON: {"risk_score": float 0.0-1.0, "risk_level": "critical|high|medium|low", "signal_count": int 1-5, "risk_factors": list of strings}

Signals:

{signals}

Knowledge:

{knowledge_chunks}

7. `backend/agents/nba_generator.py`

```

def generate(state: dict) -> dict:
    ...

```

- Calls `episodic.get_memory_context(risk_level=state["risk"]["risk_level"], risk_score=state["risk"]["risk_score"])` (P2 builds this).
- Prompts Claude with risk assessment + knowledge + memory context.
- Returns structured JSON for 3 actions (1 primary, 2 alternatives):

```

{
  "primary": {"title": "...", "description": "...", "confidence": 0.73, "estimated_impact":

```

```
"alternatives": [{...}, {...}]
}
```

- Apply confidence boost formula: `boosted_confidence = min(base_confidence (1 + 0.06 similar_cases_found), 0.97)`.
- Use `memory_context.precedent_accounts` for `precedent_accounts`.
- Store in `state["recommendations"]`.
- Log `AgentStep`: `agent_name="nba_generator"`, `action="generated_recommendations"`, `reasoning=f"Primary: {title}, confidence: {boosted}"`.

Claude prompt template:

```
You are a customer success strategist. Given the risk assessment, knowledge, and past resolved
Return ONLY JSON with keys: primary (Action), alternatives (list of 2 Actions).
Action schema: {title, description, confidence (0.0-1.0), estimated_impact, evidence_ids, prec

Risk: {risk}
Knowledge: {knowledge}
Memory: {memory_context}
```

8. Wire everything in `graph.py`

Ensure the compiled graph runs end-to-end. Test with a dummy transcript before finishing Day 2.

Day 3 — Prompt Tuning + Integration

9. Prompt tuning

Run the 3 demo scenarios (Acme Corp, Globex Corp, TechCorp) through your graph.

- Target: risk scores within $\pm 5\%$ of expected values.
- Target: primary recommendation title contains expected action keyword.
- Adjust Claude prompts if outputs are wrong. Do not change schemas.

10. Add `duration_ms` to every `AgentStep`

Use `time.time()` before and after each agent call. Calculate `(end - start) * 1000` and round to int.

11. Run eval scenarios

P2 writes `tests/eval_scenarios.py`. Run it with `pytest tests/eval_scenarios.py`.

Fix any agent producing wrong risk level or irrelevant NBA.

12. Help P3

Explain the LangGraph flow and state transitions so P3 can draw the architecture diagram.

Checklist Before Handoff

- ☐ `schemas.py` imports without errors
 - ☐ `graph.run("test transcript", "acme_corp")` returns a valid `RecommendationOutput`
 - ☐ All 5 agents log `AgentStep` with `duration_ms > 0`
 - ☐ `pytest tests/eval_scenarios.py` passes
 - ☐ No hardcoded API keys in code (use `os.getenv("ANTHROPIC_API_KEY")`)
-